

Testing Policy

Complete testing strategy for VirtualController covering unit tests, functional tests, and end-to-end tests.

Overview

VirtualController uses a multi-layer testing approach:

Layer	Scope	Tools	Target
Unit Tests	Individual components	Python unittest, manual C++	Protocol, Utilities
Functional Tests	Component interactions	Python + manual testing	Networking, Game Logic
Integration Tests	Full system flow	Manual + automated scripts	Server, Launcher, Games
E2E Tests	Complete user scenarios	Manual testing + automation	Multiplayer, Network modes

Unit Testing

Protocol Tests (Python)

Test JSON protocol parsing and serialization.

File: `tests/test_protocol.py`

Coverage:

- CONNECT message parsing
- INPUT message validation
- STATE_UPDATE deserialization
- START_GAME/RESET_GAME messages
- Malformed message handling (negative tests)

Run:

```
python tests/test_protocol.py
```

Example Cases:

- Valid CONNECT with controller count
- CONNECT missing controller field (negative)

- INPUT with invalid direction (negative)
- STATE_UPDATE with missing players array (negative)
- Oversized JSON payloads (negative)

Game Logic Tests (Manual C++)

Test core mechanics without networking.

Covered:

- Player starting positions (1-4 players)
- Food spawning logic
- Collision detection
- Score calculation
- State transitions (LOBBY → ACTIVE → ENDED)

Validation: Run single-player game and verify:

- ☐ Snake appears at correct starting position
- ☐ Food spawns randomly
- ☐ Walls stop snake
- ☐ Self-collision ends game
- ☐ End screen shows correct score

Functional Testing

Network Communication

Test server/client message exchange.

File: `tests/test_networking.py`

Coverage:

- Client connects to server on port 8765 (default)
- Server accepts connection
- CONNECT message sent
- Player ID assignment
- STATE_UPDATE broadcast
- Multi-client connections
- Connection timeout handling (negative)
- Invalid IP/port (negative)

Run:

```
# Terminal 1: Start server (listens on 8765 by default)
.\build\bin\Release\GameServer.exe

# Terminal 2: Run tests
```

```
python tests\test_networking.py -v
```

Controller Input

Test input device handling and mapping.

Covered:

- Keyboard input detection
- Joystick detection
- Dead zone application
- Multiple joystick handling (up to 4)
- Input throttling
- Invalid input rejection (negative)

Validation:

1. Connect keyboard and joysticks
2. Run each game and verify controls respond
3. Test switching between input devices

Integration Testing

Local Multiplayer

Scenario 1: 1 PC, Keyboard + 2 Joysticks

Steps:

1. Start GameServer (port 8765)
2. Start GameLibraryLauncher
3. Connect 2 joysticks to PC
4. Click Snake game
5. Verify lobby shows "3 players connected"
6. Click START GAME
7. Verify 3 snakes appear at different positions
8. Play until all die
9. Verify end screen shows 3 player colors and scores

Validation Checklist:

- ☐ Correct player count
- ☐ All 3 snakes visible
- ☐ Each responds to correct input device
- ☐ Different starting positions
- ☐ Scores calculated correctly
- ☐ BACK TO START button works

Network Multiplayer

Scenario 2: 2 PCs on same network

Setup:

- PC-A: GameServer (port 8765), GameLibraryLauncher
- PC-B: GameLibraryLauncher (connect to PC-A)

Steps:

1. Start GameServer on PC-A

```
.\build\bin\Release\GameServer.exe
```

2. Start launcher on PC-A (127.0.0.1:8765)
3. Start launcher on PC-B (PC-A IP:8765)
4. Both click Snake game
5. Verify 4 total snakes (2 per PC)
6. Both launchers show correct player counts
7. Play game

Validation Checklist:

- ☐ Network connection stable
- ☐ Player count synchronized
- ☐ All 4 snakes visible
- ☐ Collisions work across network
- ☐ Scores sync on end screen
- ☐ No latency issues

End-to-End Testing

Complete User Journey

Test Case: First-Time Player

Prerequisites:

- Windows PC with Qt 6.10+, SFML 3.0+
- Server and launcher built
- 1-2 controllers connected

Steps:

1. Open Game Library Launcher
2. See Snake in game list
3. Click Snake
4. See Lobby screen with player count
5. Click START GAME
6. Play until game over

7. Click BACK TO START
8. Play another round
9. Close game window properly

Pass Criteria:

- All steps complete without errors
- No crashes
- No visual glitches
- Controls responsive
- UI buttons clickable

Mobile Controller Flow

Test Case: Phone as Controller

Prerequisites:

- PC and phone on same network
- Mobile controller app installed
- ViGEm initialized

Steps:

1. Open Launcher WebSocket tab
2. See QR code displayed
3. Open mobile app
4. Scan QR code from launcher
5. Phone shows "Connected" message
6. Launcher shows "1/4 connected"
7. Launch Snake game
8. Phone controls player in game
9. Game ends, phone still responsive

Pass Criteria:

- ☐ QR code scans successfully
- ☐ Connection established in seconds
- ☐ Phone input registered on PC
- ☐ No lag or input loss
- ☐ Connection survives game launch/end

Negative Testing

Server Resilience

- ☐ Server continues if client disconnects mid-game
- ☐ Server doesn't crash with malformed JSON
- ☐ Server handles port already in use
- ☐ Server recovers from temporary network outage

Client Resilience

- ☐ Game continues if server briefly unavailable
- ☐ Game closes cleanly if server dies
- ☐ Game handles rapid reconnection attempts
- ☐ Multiple game launches don't leak memory

Input Validation

- ☐ Game rejects invalid direction codes
- ☐ Server rejects player ID > current players
- ☐ Malformed INPUT messages don't crash
- ☐ Oversized JSON payloads handled gracefully

Network Edge Cases

- ☐ All clients disconnect simultaneously
- ☐ One PC starts game before another connects
- ☐ Game ends while player connecting
- ☐ Network path broken during gameplay

Test Execution

Quick Test (5 minutes)

```
# Run simple protocol tests
python tests/test_protocol.py

# Manual: Single-player snake game
.\build\bin\Release\sake.exe 127.0.0.1 8765
```

Full Test Suite (30 minutes)

```
# 1. Protocol tests
python tests/test_protocol.py

# 2. Networking tests (requires server on port 8765)
# Terminal 1
.\build\bin\Release\GameServer.exe

# Terminal 2
python tests/test_networking.py -v
```

```
# 3. Manual functional testing
# Run all game configurations:
# - 1 player (keyboard)
# - 2 players (keyboard + joystick)
# - Network multiplayer (2 PCs)
```

Continuous Integration

For CI/CD pipelines:

1. Compile all targets
2. Run Python unit tests
3. Functional tests require manual validation (for now)